

← Back

Tip! Simply ask your agent directly to install the skill: <https://skills.agentlist.com/skill/mattpocock-qa-session/> Copy - it's a skill folder, with everything in it directly. easier to ingest than the listing page. ×

SKILL

mattpocock/QA Session

name: qa

name: qa

description: Interactive QA session where user reports bugs or issues conversationally, and the agent files GitHub issues. Explores the codebase in the background for context and domain language. Use when user wants to report bugs, do QA, file issues conversationally, or mentions "QA session".

QA Session

Run an interactive QA session. The user describes problems they're encountering. You clarify, explore the codebase for context, and file GitHub issues that are durable, user-focused, and use the project's domain language.

For each issue the user raises

1. Listen and lightly clarify

Let the user describe the problem in their own words. Ask **at most 2-3 short clarifying questions** focused on:

- What they expected vs what actually happened
- Steps to reproduce (if not obvious)
- Whether it's consistent or intermittent

Do NOT over-interview. If the description is clear enough to file, move on.

2. Explore the codebase in the background

While talking to the user, kick off an Agent (subagent_type=Explore) in the background to understand the relevant area. The goal is NOT to find a fix — it's to:

- Learn the domain language used in that area (check UBIQUITOUS_LANGUAGE.md)
- Understand what the feature is supposed to do
- Identify the user-facing behavior boundary

This context helps you write a better issue — but the issue itself should NOT reference specific files, line numbers, or internal implementation details.

3. Assess scope: single issue or breakdown?

Before filing, decide whether this is a **single issue** or needs to be **broken down** into multiple issues.

Break down when:

- The fix spans multiple independent areas (e.g. "the form validation is wrong AND the success message is missing AND the redirect is broken")
- There are clearly separable concerns that different people could work on in parallel
- The user describes something that has multiple distinct failure modes or symptoms

Keep as a single issue when:

- It's one behavior that's wrong in one place
- The symptoms are all caused by the same root behavior

4. File the GitHub issue(s)

Create issues with `gh issue create`. Do NOT ask the user to review first — just file and share URLs.

Issues must be **durable** — they should still make sense after major refactors. Write from the user's perspective.

For a single issue

Use this template:

What happened

[Describe the actual behavior the user experienced, in plain language]

What I expected

[Describe the expected behavior]

Steps to reproduce

1. [Concrete, numbered steps a developer can follow]
2. [Use domain terms from the codebase, not internal module names]
3. [Include relevant inputs, flags, or configuration]

Additional context

[Any extra observations from the user or from codebase exploration that help frame the

For a breakdown (multiple issues)

Create issues in dependency order (blockers first) so you can reference real issue numbers.

Use this template for each sub-issue:

Parent issue

#<parent-issue-number> (if you created a tracking issue) or "Reported during QA session"

What's wrong

[Describe this specific behavior problem – just this slice, not the whole report]

What I expected

[Expected behavior for this specific slice]

Steps to reproduce

1. [Steps specific to THIS issue]

Blocked by

```
- #<issue-number> (if this issue can't be fixed until another is resolved)
```

Or "None — can start immediately" if no blockers.

```
## Additional context
```

```
[Any extra observations relevant to this slice]
```

When creating a breakdown:

- **Prefer many thin issues over few thick ones** — each should be independently fixable and verifiable
- **Mark blocking relationships honestly** — if issue B genuinely can't be tested until issue A is fixed, say so. If they're independent, mark both as "None — can start immediately"
- **Create issues in dependency order** so you can reference real issue numbers in "Blocked by"
- **Maximize parallelism** — the goal is that multiple people (or agents) can grab different issues simultaneously

Rules for all issue bodies

- **No file paths or line numbers** — these go stale
- **Use the project's domain language** (check UBIQUITOUS_LANGUAGE.md if it exists)
- **Describe behaviors, not code** — "the sync service fails to apply the patch" not "applyPatch() throws on line 42"
- **Reproduction steps are mandatory** — if you can't determine them, ask the user
- **Keep it concise** — a developer should be able to read the issue in 30 seconds

After filing, print all issue URLs (with blocking relationships summarized) and ask: "Next issue, or are we done?"

5. Continue the session

Keep going until the user says they're done. Each issue is independent — don't batch them.

Load in your local or cloud based agent:

<https://skills.agentlist.com/skill/mattpocock-qa-session/>

Copy

Discussion

No comments yet. Be the first.

Sign in to comment

0

VOTES

▲ Vote

AUTHOR

npub1carj2jw...h8c9

SOURCE

 mattpocock

PUBLISHED

26 Apr 2026

CATEGORY

Skill

FETCHES

116